

Attaque MITM d'un service SSH et mise en place de contre-mesure

GABIN.F
21/11/2025

Objectif

- ☐ Simuler une attaque
- ☐ Identifier les vulnérabilités de SSH
- ☐ Proposer des défenses

Présentation

Adresse IP des machines virtuelles :

Intitulé de la machine	Nom de domaine pleinement qualifié	Configuration réseau	Applications et services
Serveur SSH sous Debian 10	srvssh.local.sio.fr	Adresse IPv4 : 192.168.56.10/24 Passerelle : 192.168.56.254 Serveur DNS : 127.0.0.1	Service OpenSSH port 22/TCP Service DNS Bind port 53/UDP
Client SSH sous Debian 10	clissh.local.sio.fr	Adresse IPv4 : 192.168.56.11/24 Passerelle : 192.168.56.254 Serveur DNS : 192.168.56.10	Environnement de bureau XFCE Client OpenSSH
Attaquant sous Kali Linux	NA	Adresse IPv4 : 192.168.56.12/24 Passerelle : 192.168.56.254 Serveur DNS : 192.168.56.10	Ettercap Git ssh-mitm Netfilter/Iptables
Routeur OpenBSD	rwbsd.local.sio.fr	Adresses IPv4 : em0 - DHCP em1 -192.168.56.254/24	PacketFilter

Intitulé de la machine	Nom d'utilisateur	Mot de passe
Serveur SSH sous Debian 10	etusio	Fghijkl1234*
Client SSH sous Debian 10	etusio	Fghijkl1234*
Attaquant sous Kali Linux 2020.1b	etusio	Fghijkl1234*
Routeur OpenBSD	etusio	Fghijkl1234*

Pour les commandes qui demandent des privilèges administrateurs, il faudra utiliser la commande `sudo` sous Debian GNU/Linux et `doas` sous OpenBSD.

→ `etusio@srvssh:~$ sudo service ssh restart`
`rwbsd$ doas sh /etc/netstart`

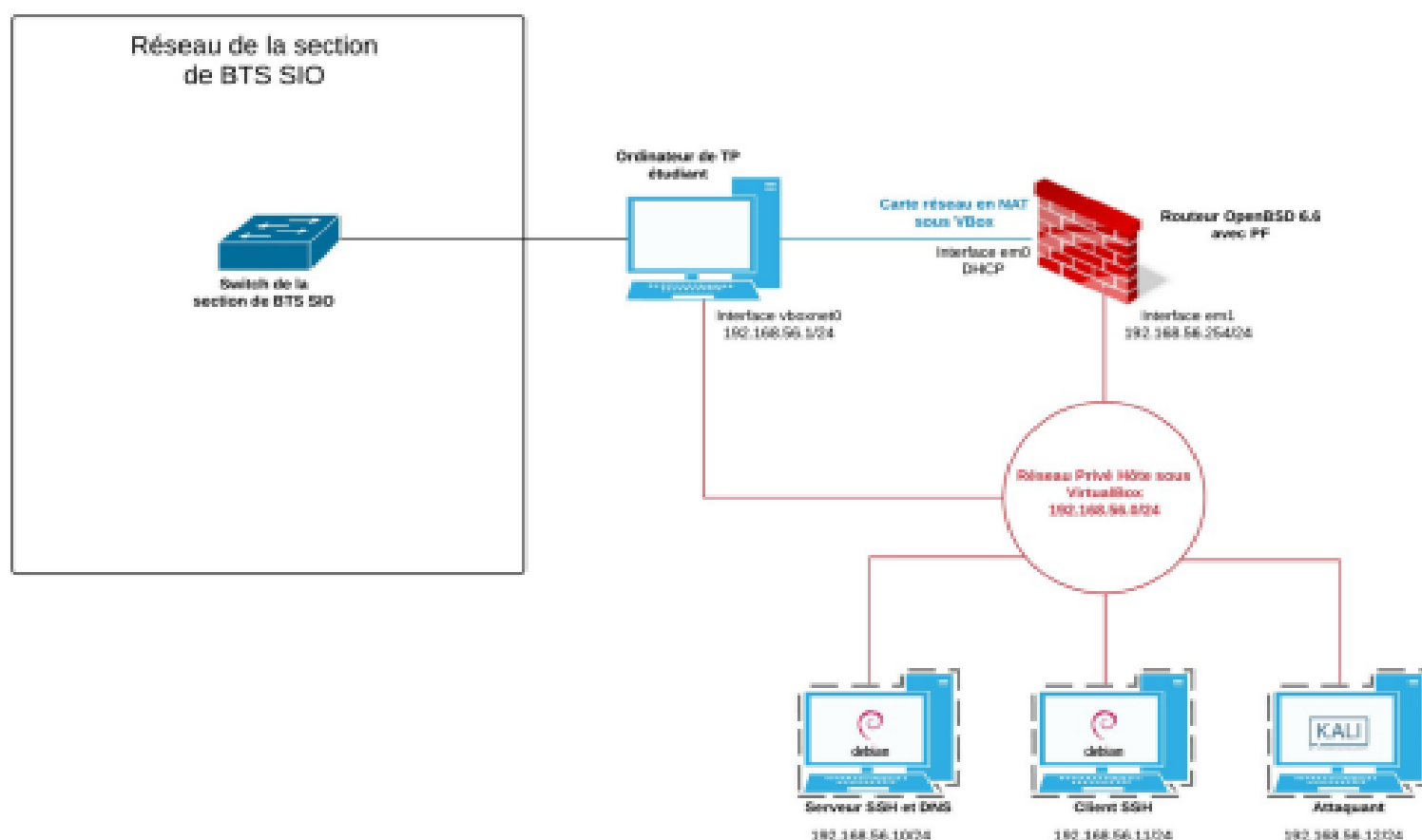
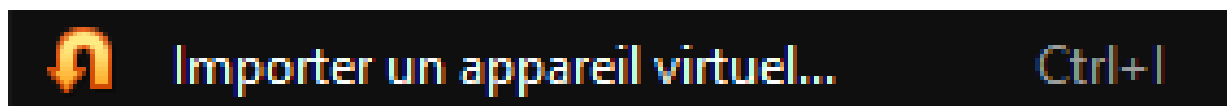


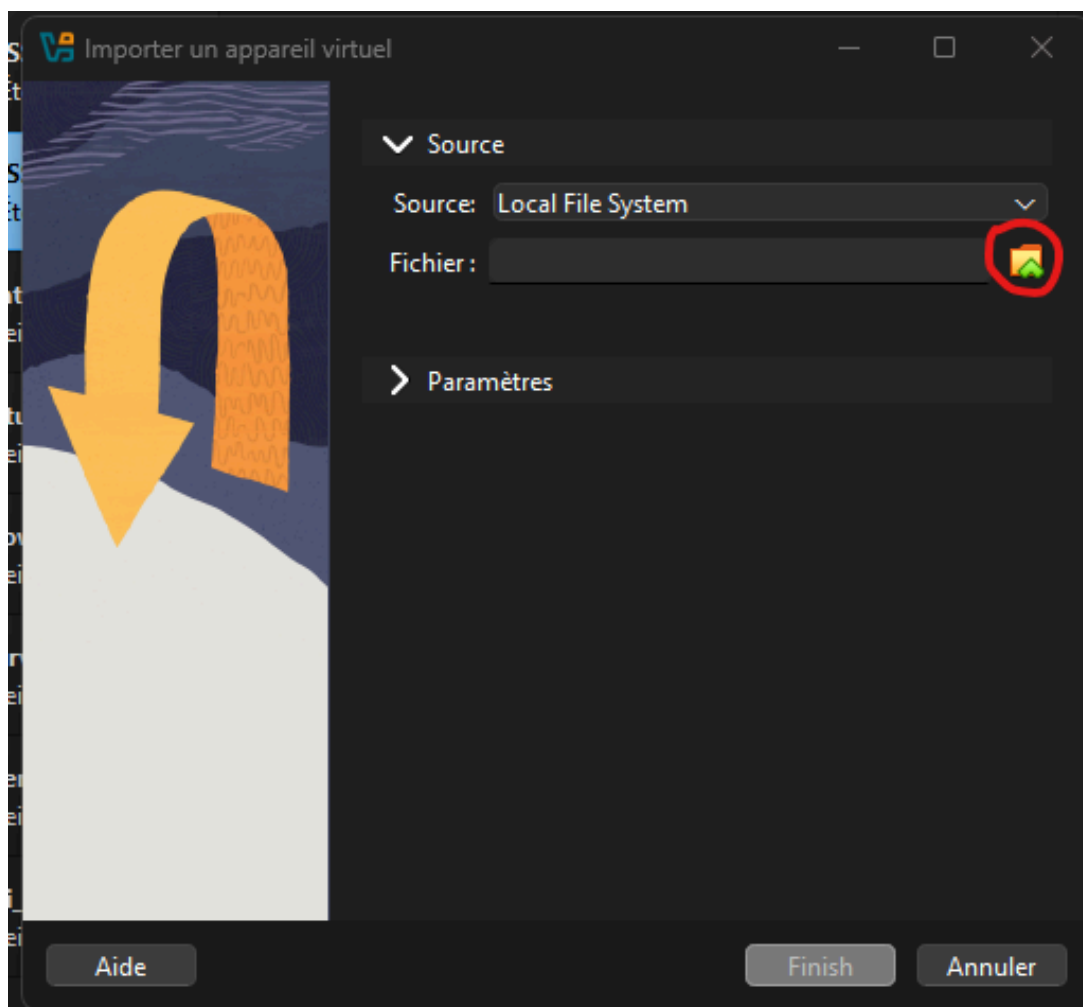
Illustration 1: Schéma logique de l'infrastructure de TP

Mise en oeuvre la maquette

Intégrez la VM en l'important sur un logiciel de gestion de système d'exploitation virtuelle :

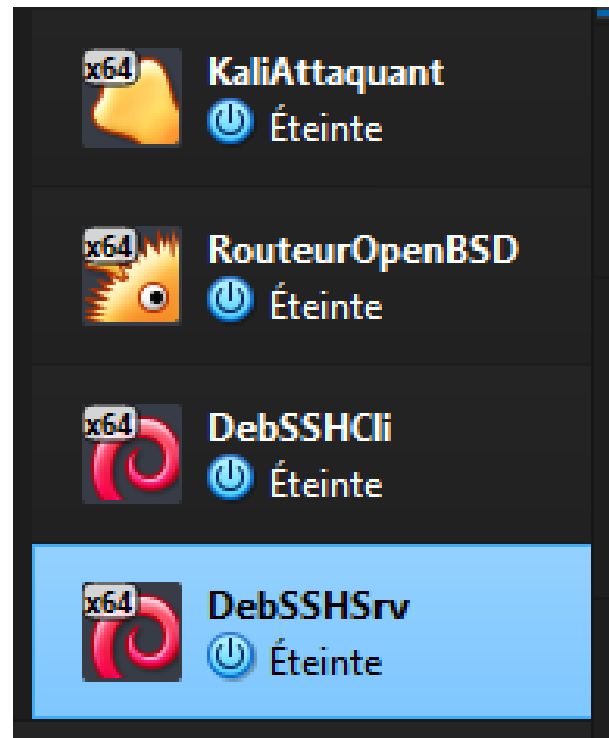


Cliquez sur le logo entouré en rouge sur l'image pour sélectionner la VM du TP :

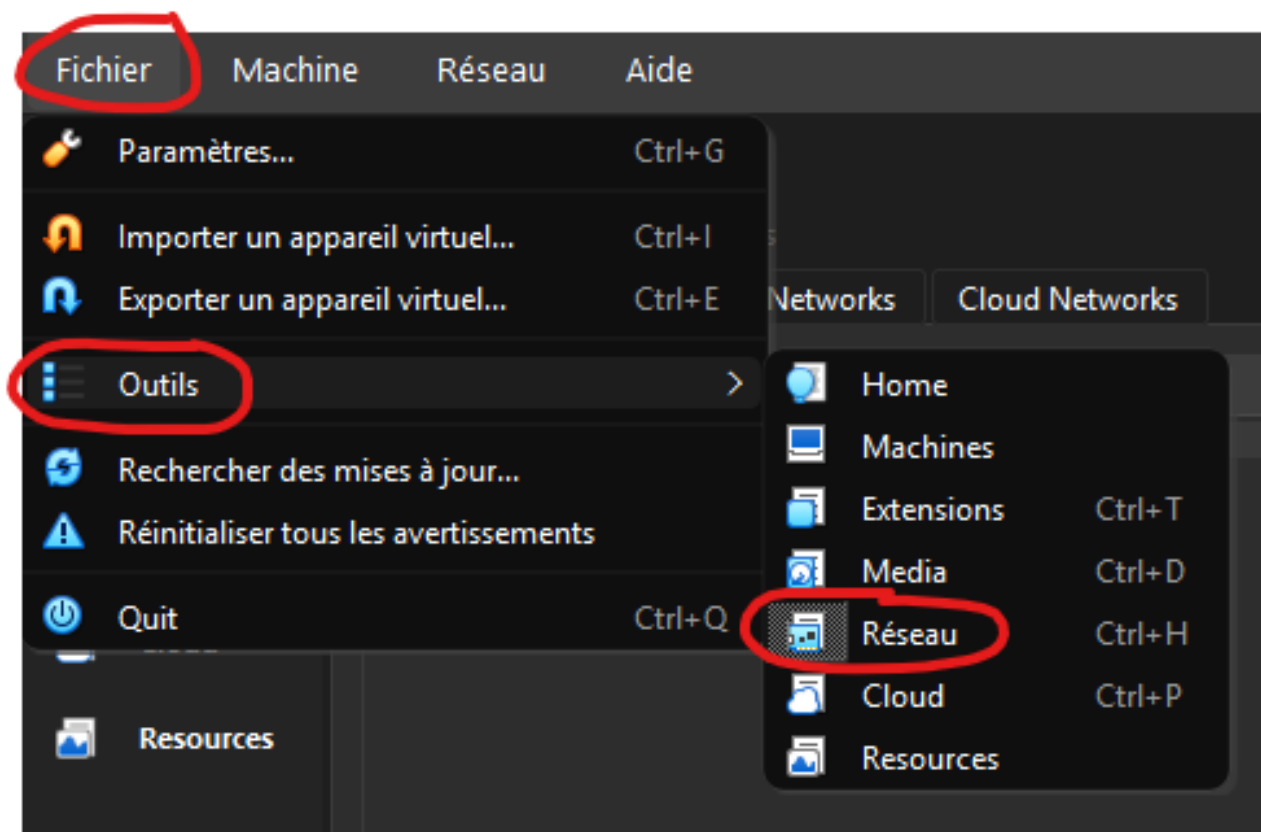


Voici l'ISO :

Vous devriez avoir 4 VM :



Vérifier que la carte réseau (vboxnet0) est bien attribué :
Fichier → Outils → Réseau



Name	IPv4 Prefix	IPv6 Prefix	DHCP Server
VirtualBox Host-Only Ethernet Adapter	192.168.56.1/24		Enabled

Allez sur l'onglet en bas de la page "Serveur DHCP" et décochez la case "Activer", ensuite appliquer les changements.

Adapter

Serveur DHCP

☐ Activer le serveur

Adresse du serveur : 192.168.56.100

Masque serveur : 255.255.255.0

Limite inférieure des adresses : 192.168.56.101

Limite supérieure des adresses : 192.168.56.254

Appliquer

Reset

Preuve de désactivation :

Name	IPv4 Prefix	IPv6 Prefix	DHCP Server
VirtualBox Host-Only Ethernet Adapter	192.168.56.1/24		Disabled

Maintenant vérifiez que les cartes réseaux sont définies en mode "réseau privé hôte".

Cliquez sur n'importe quelle machine de clic droit → configuration → onglet réseau. En cas de doute regardez l'onglet adaptateur 2 (Dans le cas du routeur).

Réseau

Adapter 1

Adapter 2

Adapter 3

Adapter 4

☒ Activer l'interface réseau

Attached to Réseau privé hôte

Name VirtualBox Host-Only Ethernet Adapter

Adapter Type Intel PRO/1000 MT Desktop (82540EM)

Promiscuous Mode Refuser

MAC Address 080027383BB2

☒ Virtual Cable Connected

Vous pourrez administrer l'ensemble de la maquette par la console VirtualBox, grâce au protocole SSH depuis votre machine hôte et d'un client natif, de putty ou kitty.

Généralités

Q1. Pourquoi l'accès aux machines virtuelles par la console ou l'interface graphique n'est pas possible avec le super-administrateur root ?

- La raison est : afin de privilégier la sécurité et se protéger de la "traçabilité", l'accès à l'utilisateur root est désactivé par **défaut** sur les distributions Linux. L'utilisateur, ayant des droits illimités, en l'espace d'une erreur, le système peut être détruit.

Q2. Expliquer à quoi sert la commande sudo et quels avantages a-t-elle sur l'utilisation de la commande su - ?

- sudo : permet d'exécuter des commandes avec les droits de root, on exploite temporairement les droits de root(administrateur) en étant un utilisateur standard.
- su - : (switch user) tu deviens le root, en changeant de session

Q3. Quelles commandes permettent de savoir si le service OpenSSH (serveur) est déjà installé et démarré ?

- Les commandes permettant de savoir si le service SSH est fonctionnel :
`sudo systemctl status ssh` / `$sudo service ssh status`, résultat → "active (running)"

```

etusio@srvssh:~$ sudo service ssh status
[sudo] Mot de passe de etusio :
• ssh.service - OpenBSD Secure Shell server
  Loaded: loaded (/lib/systemd/system/ssh.service; enabled; vendor preset: enabled)
  Active: active (running) since Fri 2025-11-28 09:03:51 CET; 5min ago
    Docs: man:sshd(8)
          man:sshd_config(5)
  Process: 423 ExecStartPre=/usr/sbin/sshd -t (code=exited, status=0/SUCCESS)
 Main PID: 450 (sshd)
    Tasks: 1 (limit: 1150)
   Memory: 4.0M
    CGroup: /system.slice/ssh.service
            └─450 /usr/sbin/sshd -D

nov. 28 09:03:51 srvssh systemd[1]: Starting OpenBSD Secure Shell server...
nov. 28 09:03:51 srvssh sshd[450]: Server listening on 0.0.0.0 port 22.
nov. 28 09:03:51 srvssh sshd[450]: Server listening on :: port 22.
nov. 28 09:03:51 srvssh systemd[1]: Started OpenBSD Secure Shell server.

```

ou encore

`$sudo dpkg -l | grep opensshserver` → est-il installé ?

ou bien

`netstat -tuln | grep 22`, résultat ! 0.0.0.0:22 en état "LISTEN"

Q4. Indiquer le répertoire où sont stockées les clés publique et privée créées ainsi que le positionnement des permissions appliquées sur les fichiers correspondants. Puis indiquer quel est le fichier de configuration du service SSH.

→ Les chemins des fichiers des clés :

→ fichier de configuration du service SSH :

`/etc/ssh/sshd_config`

→ répertoire des clés d'hôte : `/etc/ssh/ssh_host_*` → nom du fichier en question en vert.

→ clés créées :

publiques : `ssh_host_ecdsa_key.pub`

privées : `ssh_host_ecdsa_key`

Les clés privées doivent être en permission : **600**

DebSSHCli :

```
etusio@clissh:~$ sudo systemctl status ssh
[sudo] Mot de passe de etusio :
sudo: systemctl : commande introuvable
etusio@clissh:~$ ls -l /etc/ssh/
moduli
ssh_config
sshd_config
ssh_host_ecdsa_key
ssh_host_ecdsa_key.pub
ssh_host_ed25519_key
ssh_host_ed25519_key.pub
ssh_host_rsa_key
ssh_host_rsa_key.pub
etusio@clissh:~$
```

DebSShSrv :

```
etusio@srvssh:~$ ls -l /etc/ssh/
total 584
-rw-r--r-- 1 root root 565189 janv. 31 2020 moduli
-rw-r--r-- 1 root root 1580 janv. 31 2020 ssh_config
-rw-r--r-- 1 root root 3250 janv. 31 2020 sshd_config
-rw----- 1 root root 505 mars 29 2020 ssh_host_ecdsa_key
-rw-r--r-- 1 root root 173 mars 29 2020 ssh_host_ecdsa_key.pub
-rw----- 1 root root 399 mars 29 2020 ssh_host_ed25519_key
-rw-r--r-- 1 root root 93 mars 29 2020 ssh_host_ed25519_key.pub
-rw----- 1 root root 1823 mars 29 2020 ssh_host_rsa_key
-rw-r--r-- 1 root root 393 mars 29 2020 ssh_host_rsa_key.pub
etusio@srvssh:~$ _
```

Petit bilan des notions

- Le fichier se terminant en key et key.pub sont les paires de clés publique et privée du serveur ssh. Le serveur dispose de plusieurs paires de clés en fonction du choix de l'algorithme par le client.
- Le fichier ssh **sshd_config** permet de définir les paramètres de configuration du serveur.
- Le fichier **ssh_config** s'affiche permet de définir les paramètres du client ssh
- Le fichier moduli sert à l'échange de clé **Diffie-Hellman** lors de l'établissement d'une connexion entre un client et un serveur.

Première utilisation (Authentification par mot de passe)

Q5. Que signifie cette alerte qui est affichée à l'écran ? Devez-vous continuer l'opération ? Pourquoi ?

→ La machine cliente ne connaît pas ce serveur . Le message demande une confirmation pour enregistrer la clé publique.

Tester la commande :

```
etusio@clissh:~$ ssh etusio@srvssh.local.sio.fr
The authenticity of host 'srvssh.local.sio.fr (192.168.56.10)' can't be established.
ECDSA key fingerprint is SHA256:lPIxEKxsYHPP3i7iMiZZXLYUoW9viLwSfF39MNoWLM4.
Are you sure you want to continue connecting (yes/no)?
```

Après avoir dis "yes" :

```
Warning: Permanently added 'srvssh.local.sio.fr,192.168.56.10' (ECDSA) to the l
st of known hosts.
etusio@srvssh.local.sio.fr's password:
Linux srvssh 4.19.0-8-amd64 #1 SMP Debian 4.19.98-1+deb10u1 (2020-04-27) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Fri Nov 28 09:04:40 2025
```

Cela signifie que c'est la première fois qu'on se connecte sur cette machine, à l'aide du protocole SSH, On doit accepter de poursuivre uniquement si on est en mesure de vérifier que l'empreinte de la clé présentée est bien celle de l'autre, sur lequel on cherche à se connecter. Pour cela, on aurait dû nécessairement récupérer cette empreinte au préalable sur le serveur en question (Le serveur prend l'empreinte). Ça n'a pas été en mesure de vérifier cette empreinte, on est en incapacité de déterminer l'identité de l'hôte.

Q6. Lors d'une prochaine connexion depuis le même client sur ce serveur, ce message apparaîtra-t-il à nouveau ? Pourquoi ?

Normalement, non, car l'empreinte a bien été enregistrée du côté du serveur, et celui-ci le reconnaîtra aux prochaines connexions à venir.

Si l'on valide l'identité, le client ssh est responsable du domaine ou l'adresse IP. Il considère par la suite qu'il n'est plus nécessaire d'afficher ce message car le serveur a accordé sa confiance à l'hôte en lui transmettant ces données.

Q7. Sur la machine virtuelle cliente, expliquer à quoi sert le fichier /home/etusio/.ssh/known_hosts

→

Ici vérifier que vous n'êtes pas dans le serveur : **ERREUR A**
NE PAS FAIRE

“

```
etusio@srvssh:~$ ssh-keygen
Generating public/private rsa key pair.
Enter file in which to save the key (/home/etusio/.ssh/id_rsa):
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/etusio/.ssh/id_rsa.
Your public key has been saved in /home/etusio/.ssh/id_rsa.pub.
The key fingerprint is:
SHA256:air85CMSlvng7VjwGNxv/unabNms8PMKRMVGLAoFQQA etusio@srvssh
The key's randomart image is:
+---[RSA 2048]---+
|E.o++.  +o      |
|  .    ..+      |
|  .    ..o      |
|  .    ...      |
|  .+..  .S      |
|.. += o.        |
|.o..=oo=  +     |
|. +++=o  *+.o   |
|. ==o  o=X=.    |
+-----[SHA256]-----+
```

```
etusio@srvssh:~$ ls -a ~/.ssh
.  ..  id_rsa  id_rsa.pub
```

```
etusio@srvssh:~$ echo > ~/.ssh/known_hosts
```

Le fichier n'a **aucun octet** :

```
etusio@srvssh:~$ ls -l ~/.ssh/known_hosts
-rw-r--r-- 1 etusio etusio 1 nov. 28 10:08 /home/etusio/.ssh/known_hosts
etusio@srvssh:~$
```

jusqu'ici “

etusio@clissh :

écrire : cat .ssh/known_hosts pour voir le fichier de réception des clés

```
etusio@clissh:~$ cat .ssh/known_hosts
|1|7udFds9H5nExHKxnJLBKAHVRel8=|rYhmvUNPiKbKtJGaJhBjrIHg/dY= ecdsa-sha2-
nistp256 AAAAE2VjZHNhLXNoYTItbmlzdHAyNTYAAAAIbmlzdHAyNTYAAABBBKBiccKG7qS
9Tmm7DkTS1/HU2k7qtg1oRAg+fb4ejK08+K1q4Ypr7mnNFXu7SX3SJe5jCf9S4M/itNiGcPw
D82o=
|1|c3tjs152admG/p32nKUr+Yqmn2w=|m3egKduqTmLC1vlxMEDgeuV8iJQ= ecdsa-sha2-
nistp256 AAAAE2VjZHNhLXNoYTItbmlzdHAyNTYAAAAIbmlzdHAyNTYAAABBBKBiccKG7qS
9Tmm7DkTS1/HU2k7qtg1oRAg+fb4ejK08+K1q4Ypr7mnNFXu7SX3SJe5jCf9S4M/itNiGcPw
D82o=
```

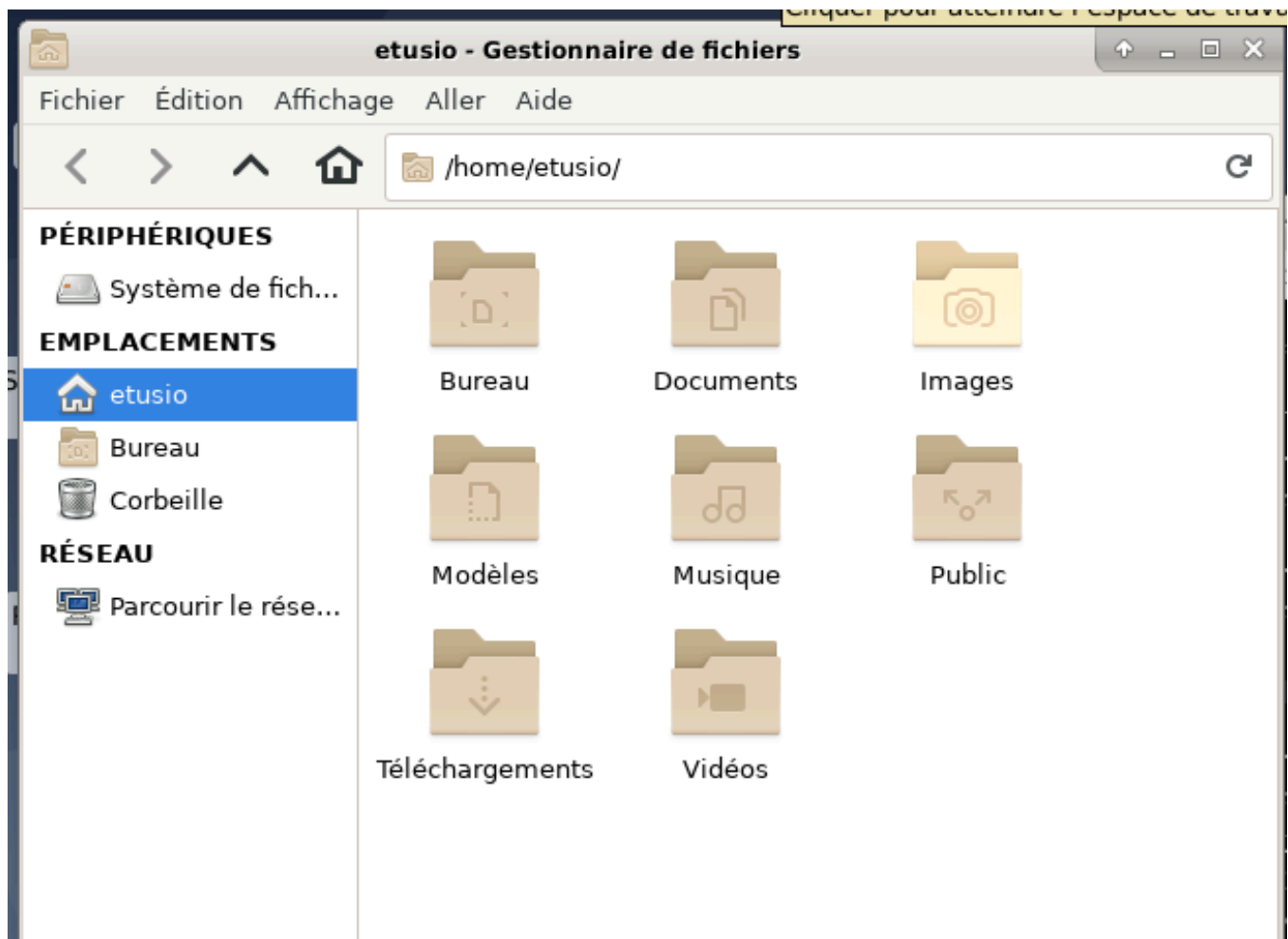
écrire : ssh-keygen -F srvssh.local.sio.fr

```
etusio@clissh:~$ ssh-keygen -F srvssh.local.sio.fr
# Host srvssh.local.sio.fr found: line 1
|1|7udFds9H5nExHKxnJLBKAHVRel8=|rYhmvUNPiKbKtJGaJhBjrIHg/dY= ecdsa-sha2-
nistp256 AAAAE2VjZHNhLXNoYTItbmlzdHAyNTYAAAAIbmlzdHAyNTYAAABBBKBiccKG7qS
9Tmm7DkTS1/HU2k7qtg1oRAg+fb4ejK08+K1q4Ypr7mnNFXu7SX3SJe5jCf9S4M/itNiGcPw
D82o=
```

écrire echo → ~/.ssh/known_hosts pour vider le contenu

```
etusio@clissh:~$ echo > ~/.ssh/known_hosts
```

Preuve que le fichier n'est plus présent :



Cette commande est un fichier qui stock les clés publiques

Lors de la prochaine connexion depuis le client sur ce client, ce message n'apparaîtra plus car la clé a été créée. Cela signifie que lors de la question au ssh, le client compare les clés contenues dans le fichier "known_hosts". La clé est présente donc le fichier, le message n'apparaîtra pas

Découverte des hôtes et services présents sur un réseau local

Q8. Indiquer quelles sont les informations que peut obtenir un attaquant grâce à ces commandes ?

```
etusio@clissh:~$ ssh-keyscan 192.168.56.10
# 192.168.56.10:22 SSH-2.0-OpenSSH_7.9p1 Debian-10+deb10u2
192.168.56.10 ecdsa-sha2-nistp256 AAAAE2VjZHNhLXNoYTItbmlzdHAyNTYAAAAIbmlzdHAyNTYAAABBBKBIccKG7qS9Tmm7DkTS1/HU2k7qtg1oRAG+fb4ejK08+K1q4Ypr7mnNFXu7SX3SJe5jCf9S4M/itNiGcPwD82o=
# 192.168.56.10:22 SSH-2.0-OpenSSH_7.9p1 Debian-10+deb10u2
192.168.56.10 ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAQAC8aA9rVXabSZVHR+wHNSkny7AslAlG4k66bpRwLKe0JyDsIVmTYkjwSteGF6HbxBJhX/kUToq047mHxVaGMzn9ahcJOYj8lWp9FExW2nlm8nHX4BxsB4irZh9cIIBf/hegClxoDwJOdNVQbCUDzjMS8WfEIu58QaugVRczpRAGhamfmFNfgHSIJLGuI/kdurEaetbNckVEVEkZLj5Ic6jV0FW760ci4Q4J0uCmMUEnC2bZBzKERTCYtggnK+us2jRF1DmWxU04gdGiHxTJY+d/aFYnxYMQsloowEmCpNLjHuimveii4kPASjQDNBcwA/GXx7oWtEnJrX69y00DzVTl
# 192.168.56.10:22 SSH-2.0-OpenSSH_7.9p1 Debian-10+deb10u2
192.168.56.10 ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAI0WKP++8alJ810b+o0Bg7v5Jlzyj60urm0WMhKA+dgBE
```

Le nmap permet de réaliser un scan de différent adresse IP, de ports sélectionner, puis afficher les ports utilisé ainsi que la version des services utilisée, de la version de service à l'écoute de ces ports le scan permet à l'attaquant de récolter des informations, connaître les services utilisés Il est ainsi possible à l'aide d'outils spécialisés, comme metasploit d'obtenir les différentes failles potentielles liées à une version précise d'un service comme ssh

Scanner les trois hôtes : .254, .11, .10

```
etusio@kali:~$ nmap -sV 192.168.56.11
Starting Nmap 7.80 ( https://nmap.org ) at 2025-11-28 11:36 CET
Nmap scan report for clissh.local.sio.fr (192.168.56.11)
Host is up (0.0026s latency).
Not shown: 999 closed ports
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 7.9p1 Debian 10+deb10u2 (protocol 2.0)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 0.66 seconds
```


Simulation d'une attaque de l'homme du milieu entre votre client et votre serveur SSH

Q9.Expliquer les principes généraux d'une attaque de l'homme du milieu (Man in the Middle).

L'attaque de l'homme du milieu consiste pour un attaquant à récupérer les trafics circulant entre deux hôtes du client entre deux

Si la personne malveillante réalise une attaque.. son but est d'écouter ce qu'il se passe afin de récupérer les informations. Elle porte uniquement atteinte à la confidentialité des échanges. Dans le cas d'une attaque MITM active, l'attaquant peut également modifier le contenu des données échangées. Il porte ainsi atteinte à la confidentialité et l'intégrité des informations échangés.Voici en quoi consiste une attaque MITM

Q10.Noter les associations adresse IP / adresse MAC présentes sur les deux machines. Sont-elles cohérentes ?

etusio@clissh :

```
etusio@clissh:~$ ip neigh show
192.168.56.12 dev enp0s3 lladdr 08:00:27:38:3b:b2 STALE
192.168.56.10 dev enp0s3 lladdr 08:00:27:41:16:10 STALE
```

etusio@srvssh :

```
etusio@srvssh:~$ ip neigh show
192.168.56.12 dev enp0s3 lladdr 08:00:27:38:3b:b2 STALE
192.168.56.254 dev enp0s3 lladdr 08:00:27:4a:97:13 STALE
192.168.56.11 dev enp0s3 lladdr 08:00:27:e5:2d:33 REACHABLE
etusio@srvssh:~$
```


Il y a eu une adresse ip en trop 192.168.56.12 dans les deux machines (serveur et cliente) Qui est en commun, cette adresse est celle de l'intrus.

Dans la liste en joint cette tâche ARP et les configurations réseau sur le client et sur le serveur. Permet d'affirmer que ces association sont totalement cohérentes.

Client :

```
etusio@clissh:~$ ip addr show enp0s3
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP group default qlen 1000
    link/ether 08:00:27:e5:2d:33 brd ff:ff:ff:ff:ff:ff
    inet 192.168.56.11/24 brd 192.168.56.255 scope global enp0s3
        valid_lft forever preferred_lft forever
    inet6 fe80::a00:27ff:fee5:2d33/64 scope link
        valid_lft forever preferred_lft forever
```

Serveur :

```
etusio@srvssh:~$ ip addr show enp0s3
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP group default qlen 1000
    link/ether 08:00:27:41:16:10 brd ff:ff:ff:ff:ff:ff
    inet 192.168.56.10/24 brd 192.168.56.255 scope global enp0s3
        valid_lft forever preferred_lft forever
    inet6 fe80::a00:27ff:fe41:1610/64 scope link
        valid_lft forever preferred_lft forever
etusio@srvssh:~$
```

Q11.Pourquoi l'activation du routage sur la machine de l'attaquant est indispensable au bon fonctionnement de l'attaque MITM ?

L'activation du routeur sur la machine de l'attaquant est indispensable pour une attaque MITM car elle permet à l'attaquante de rediriger le trafic, modifier les données en transit, écouter discrètement et ré-acheminer les réponses ce qui est essentiel pour mener à bien l'attaque

Q12. Pourquoi cette redirection de ports est indispensable au succès de l'attaque de l'homme du milieu ?

La redirection de port est indispensable au succès de l'attaque mitm car elle permet à l'attaquant d'intercepter, d'analyser et de manipuler le trafic entre la victime et la cible tout en restant invisible pour la victime.

Le logiciel ssh-mitm écoute sur le port 2222 afin d'être sûr de ne pas perturber un éventuel service ssh déjà existant sur la machine de l'attaquant. La commande iptables est une redirection de port (NAT port forwarding), elle consiste à dire au système kali que toutes les connexions qu'il reçoit sur son port 22 doivent être redirigées vers l'outil ssh-mitm.

Mise en place d'une attaque ARP Spoofing afin d'obliger le client et le serveur SSH à faire transiter les trames Ethernet échangées par l'attaquant.

Q13. Indiquer en quoi une attaque de type ARP Spoofing peut être utile ici au pirate ?

L'attaque ARP spoofing (ou ARP poisoning) consiste à utiliser les failles d'implémentation du protocole ARP afin d'empoisonner le cache ARP d'une machine. Le but de cette manœuvre est de se faire passer pour un autre hôte présent sur le réseau auprès de la machine que nous avons ciblée.

Ainsi grâce à cette manipulation, le pirate pourra empoisonner le cache ARP des victimes afin de les obliger à faire transiter les trames des machines, (usurpation de l'adresse MAC du serveur auprès du client et inversement)

Q14. Comparer les caches ARP du client et du serveur avec les associations notées précédemment lors de la question 10. Qu'en concluez-vous ?

Avant Machine cliente

```
etusio@srvssh:~$ ip neigh show
192.168.56.254 dev enp0s3 INCOMPLETE
192.168.56.11 dev enp0s3 lladdr 08:00:27:2e:82:f6 REACHABLE
192.168.56.12 dev enp0s3 lladdr 08:00:27:e2:3f:cf STALE
```

Après

```
etusio@clissh:~$ ip neigh show
192.168.56.10 dev enp0s3 lladdr 08:00:27:1c:91:c0 STALE
etusio@clissh:~$
```

```
etusio@clissh:~$ ip addr show enp0s3
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP
group default qlen 1000
    link/ether 08:00:27:2e:82:f6 brd ff:ff:ff:ff:ff:ff
    inet 192.168.56.11/24 brd 192.168.56.255 scope global enp0s3
        valid_lft forever preferred_lft forever
    inet6 fe80::a00:27ff:fe2e:82f6/64 scope link
        valid_lft forever preferred_lft forever
etusio@clissh:~$
```

```
etusio@clissh:~$ ip neigh show
192.168.56.10 dev enp0s3 lladdr 08:00:27:1c:91:c0 STALE
etusio@clissh:~$
```

Avant du Serveur Kali

```
etusio@srvssh:~$ ip neigh show
192.168.56.254 dev enp0s3 INCOMPLETE
192.168.56.11 dev enp0s3 lladdr 08:00:27:2e:82:f6 STALE
192.168.56.12 dev enp0s3 lladdr 08:00:27:e2:3f:cf STALE
etusio@srvssh:~$
```

Après

```
etusio@srvssh:~$ ip neigh show
192.168.56.254 dev enp0s3 INCOMPLETE
etusio@srvssh:~$ ip neigh show
192.168.56.254 dev enp0s3 INCOMPLETE
etusio@srvssh:~$ _
```

```
etusio@srvssh:~$ ip addr show enp0s3
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP group default qlen 1000
    link/ether 08:00:27:1c:91:c0 brd ff:ff:ff:ff:ff:ff
    inet 192.168.56.10/24 brd 192.168.56.255 scope global enp0s3
        valid_lft forever preferred_lft forever
    inet6 fe80::a00:27ff:fe1c:91c0/64 scope link
        valid_lft forever preferred_lft forever
```

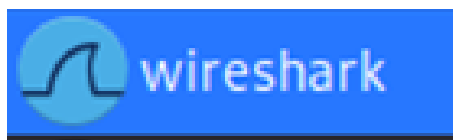
```
etusio@kali:~$ ip addr show eth0
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP group default qlen 1000
    link/ether 08:00:27:e2:3f:cf brd ff:ff:ff:ff:ff:ff
    inet 192.168.56.12/24 brd 192.168.56.255 scope global noprefixroute eth0
        valid_lft forever preferred_lft forever
    inet6 fe80::63d6:aea2:db9d:f7e1/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
```

Après comparaison avec les associations relevées lors de la question 10 il est possible d'affirmer que les caches ARP du client et du serveur ne contiennent plus les mêmes informations. Ainsi, l'adresse IP 192.168.56.10 faisant référence au serveur ssh sur le client correspond à l'adresse mac de l'attaquant (kali). Il envoie de même sur le serveur ssh ou l'adresse IP 192.168.56.11 du client faisant référence au serveur

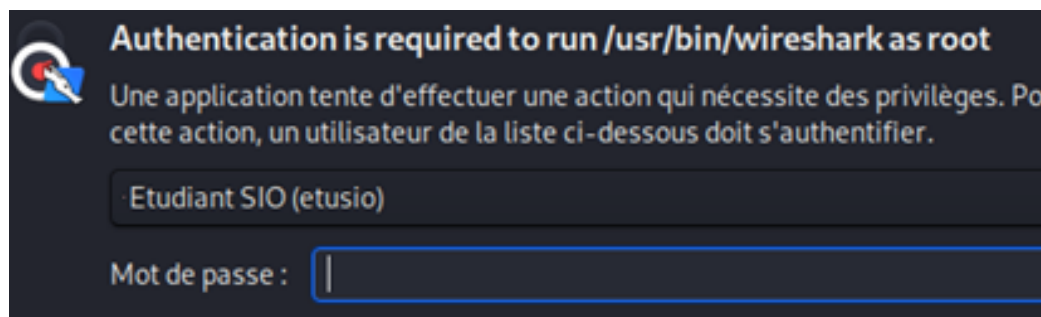
correspond également à l'adresse mac de l'attaquant. Nous pouvons affirmer qu'un empoisonnement des tâches ARP est en cours et que les 2 machines sont probablement victimes d'une attaque de type ARP poisoning.

Q15.À partir de ces différentes observations, expliquez en détails comment fonctionne une attaque ARP Spoofing.

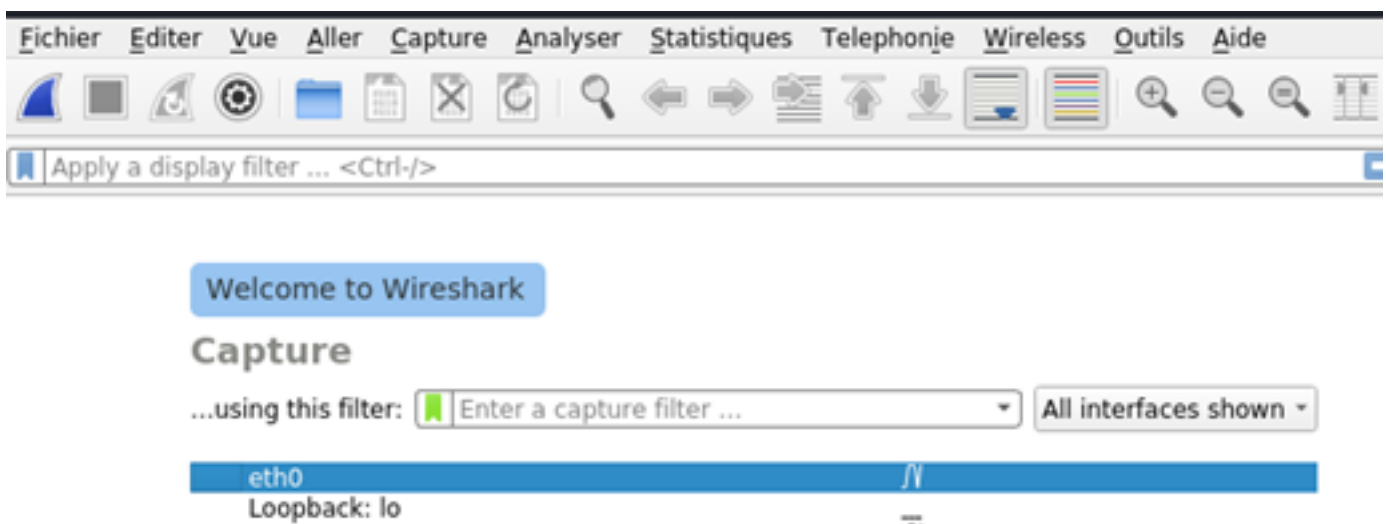
Pour faire une attaque ARP Spoofing sur le Serveur Kali, nous devons aller sur l'icône de la machine en "bleu", puis rechercher le logiciel wireshark.



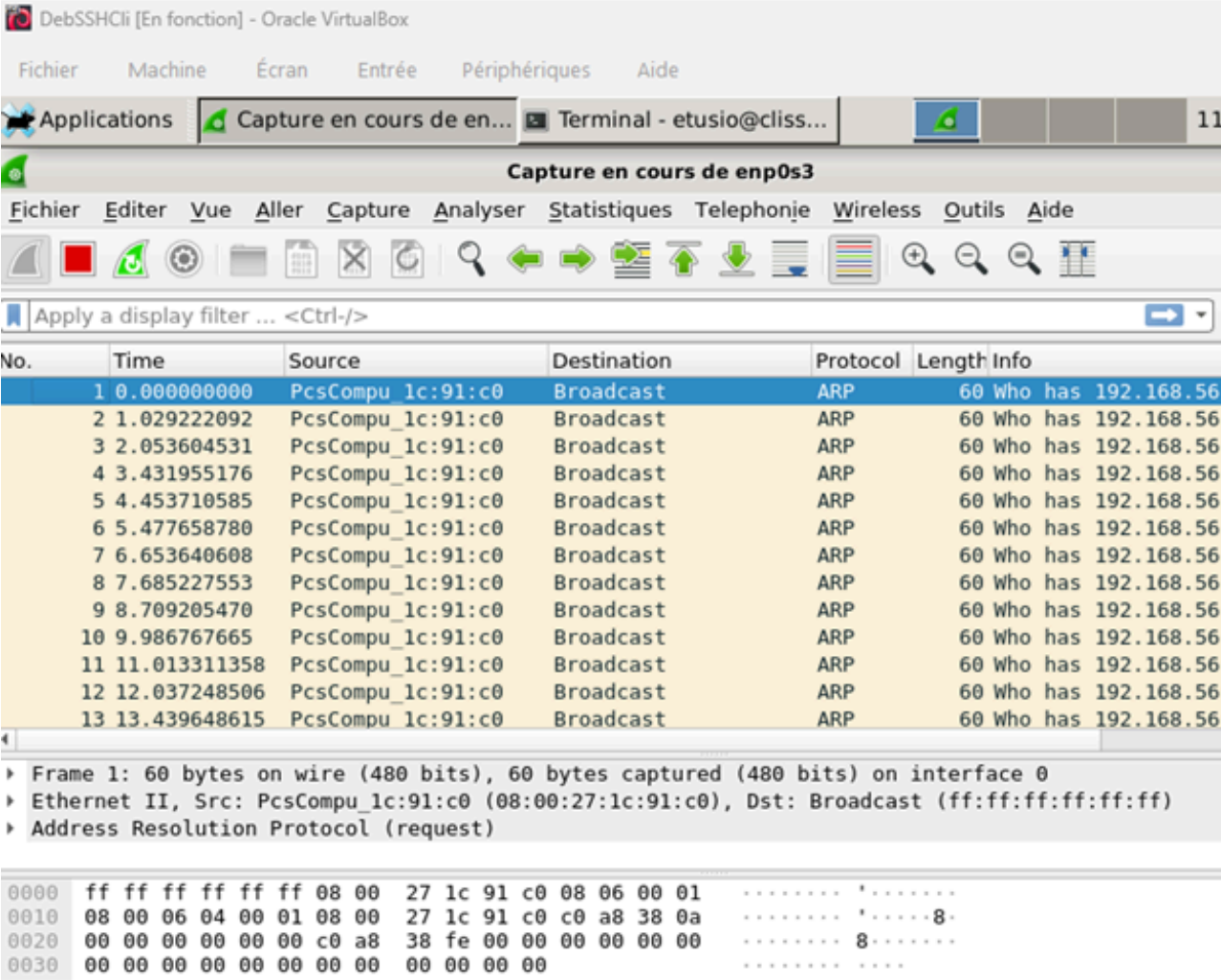
Ensuite il faut s'authentifier en mettant le mot de passe



Ensuite, sur cette page, cliquer sur "eth0"



Juste après nous pourrons voir les trames du logiciel wireshark



DebSSHCLI [En fonction] - Oracle VirtualBox

Fichier Machine Écran Entrée Périphériques Aide

Applications Capture en cours de en... Terminal - etusio@cliss... 11

Capture en cours de enp0s3

Fichier Editer Vue Aller Capture Analyser Statistiques Telephonie Wireless Outils Aide

Apply a display filter ... <Ctrl-/>

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	PcsCompu_1c:91:c0	Broadcast	ARP	60	Who has 192.168.56
2	1.029222092	PcsCompu_1c:91:c0	Broadcast	ARP	60	Who has 192.168.56
3	2.053604531	PcsCompu_1c:91:c0	Broadcast	ARP	60	Who has 192.168.56
4	3.431955176	PcsCompu_1c:91:c0	Broadcast	ARP	60	Who has 192.168.56
5	4.453710585	PcsCompu_1c:91:c0	Broadcast	ARP	60	Who has 192.168.56
6	5.477658780	PcsCompu_1c:91:c0	Broadcast	ARP	60	Who has 192.168.56
7	6.653640608	PcsCompu_1c:91:c0	Broadcast	ARP	60	Who has 192.168.56
8	7.685227553	PcsCompu_1c:91:c0	Broadcast	ARP	60	Who has 192.168.56
9	8.709205470	PcsCompu_1c:91:c0	Broadcast	ARP	60	Who has 192.168.56
10	9.986767665	PcsCompu_1c:91:c0	Broadcast	ARP	60	Who has 192.168.56
11	11.013311358	PcsCompu_1c:91:c0	Broadcast	ARP	60	Who has 192.168.56
12	12.037248506	PcsCompu_1c:91:c0	Broadcast	ARP	60	Who has 192.168.56
13	13.439648615	PcsCompu_1c:91:c0	Broadcast	ARP	60	Who has 192.168.56

Frame 1: 60 bytes on wire (480 bits), 60 bytes captured (480 bits) on interface 0

Ethernet II, Src: PcsCompu_1c:91:c0 (08:00:27:1c:91:c0), Dst: Broadcast (ff:ff:ff:ff:ff:ff)

Address Resolution Protocol (request)

```
0000  ff ff ff ff ff ff 08 00 27 1c 91 c0 08 06 00 01  .....*. ....
0010  08 00 06 04 00 01 08 00 27 1c 91 c0 c0 a8 38 0a  .....8. ....
0020  00 00 00 00 00 00 c0 a8 38 fe 00 00 00 00 00 00  .....8. ....
0030  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ..... .....
```

Le protocole ARP ne dispose d'aucun mécanisme d'authentification . Ainsi lorsqu'une machine reçoit une requête arp (ARP reply) lui signifiant que tel adresse ip correspond à telle adresse mac, cette dernière prend cette info comme argent comptant et la stock dans son cache arp sans vérifier si l'émetteur de cette trame ARP est légitime .

Dans le cas présent, l'attaquant émet à intervalle régulier des trames ARP reply forgé auprès du client et du serveur ssh :

-il envoie des trames ARP reply au client lui affirmant que l'adresse IP 192.168.56.10 correspond a l'adresse mac de l'attaquant

-il envoi des trames ARP reply au serveur lui affirmant que l'adresse IP 192.168.156.11 correspond a l'adresse mac de l'attaquant

Ainsi lorsque l'on observe les caches ARP présent sur le client et le serveur ssh, on constate que cette fausse info a effectivement bien été enregistrée dans le cache ARP. C requête est émise à intervalle régulier pour s'assurer que le cache arp des 2 victimes ne soit pas entre temps modifier par une requête légitime émanant du client ou du serveur

Ce type d'attaque sur le réseau est assez visible et il existe un certain nombre de mesures pour y faire face. L'adm réseau peut superviser le fonctionnement du protocole ARP sur son réseau à l'aide de l'outil ARP Watch afin d'être informer des irrégularités qui pourrait survenir installer un ips ou mettre en œuvre une authentification 802.1x

Q16.Envoyer une requête ping (icmp-écho) depuis le client vers le serveur (192.168.56.10). Puis vérifier à l'aide d'une capture de trame sur la machine Kali Linux que ces dernières passent effectivement bien par l'attaquant. Quels éléments démontrent que l'attaque se déroule correctement ?

Si l'attaque se déroule normalement le client doit bien recevoir une réponse à sa requête de la part du serveur, de son côté l'attaquant doit voir dans wireshark la requête icmp echo entre les 2 victimes, il passe désormais obligatoirement par lui.

Q17.Recopier la ligne qui contient le login et le mot de passe capturés dans le fichier /var/log/auth.log

Q18.Que contient le fichier /home/ssh-mitm/shell_session_0.txt présent sur Kali Linux ? NB : L'accès à ce fichier nécessite l'utilisation de la commande sudo.

```
etusio@kali:~$ sudo /home/ssh-mitm/shell_session_0.txt
[sudo] Mot de passe de etusio :
sudo: /home/ssh-mitm/shell_session_0.txt : commande introuvable
etusio@kali:~$
```

Q19.Expliquer pourquoi ce message d'erreur apparaît ?

Lorsque le client essaye de se reconnecter après l'attaque, ssh nous prévient que l'empreinte de clé proposée par le serveur n'est pas la même que celle qu'il a acceptée lors de sa connexion précédente. Ce message met en garde contre une possible tentative d'attaque mitm.

Il faut supprimer le message dans `/home/ssh/known_hosts`

Q20. Proposer une solution afin de pouvoir à nouveau se connecter au service SSH depuis le client.

Il faut supprimer le message dans `/home/ssh/known_hosts`

Q21.

L'authentification par double authentification, sinon par clé de chiffrement, possède une double authentification et surtout `ecdsa` permet de générer des tailles de clé plus petites (256 à 512 bits).

Renforcement de la sécurité du service OpenSSH et remédiation

1. Mise en place d'une authentification par clés de chiffrement

Créer la clé de chiffrement : `ssh-keygen -b 256 -t ecdsa`
mot de passe : `TPMITMcerta`


```

etusio@clissh:~$ sudo service ssh restart
etusio@clissh:~$ ssh-keygen -b 256 -t ecdsa
Generating public/private ecdsa key pair.
Enter file in which to save the key (/home/etusio/.ssh/id_ecdsa):
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/etusio/.ssh/id_ecdsa.
Your public key has been saved in /home/etusio/.ssh/id_ecdsa.pub.
The key fingerprint is:
SHA256:jX3LZt5fh28HhrItlea1H7gmMATApMYClCjKw8Y1qHQ etusio@clissh
The key's randomart image is:

```

```

The key's randomart image is:
+---[ECDSA 256]---+
|o.o ...          |
|++ E o .         |
|X o + .          |
|oB + =           |
|. = S o .o       |
|      o.o=.=.    |
|      oB=+.+o    |
|      o=ooo.*    |
|      .+.+=     |
+-----[SHA256]-----+

```

Q22.A votre avis, pourquoi la mise en place de cette phrase de chiffrement pour accéder à la clé privée est extrêmement importante ?

Il est essentiel d'établir une phase de chiffrement pour sécuriser l'accès à la clé C2, ce qui contribue à prévenir les accès non autorisés, à garantir la confidentialité des données, à contrecarrer les attaques brutes et à conférer un contrôle personnel sur la clé publique. Cette mesure revêt une importance capitale.

En exécutant la commande :

clé privée **\$HOME/.ssh/id_ecdsa**

clé publique **\$HOME/.ssh/id_ecdsa.pub**,

Qui peut être connu par tout le monde. Elle est importante car, elle permet de garantir l'intégrité du client lors de connexion

Q23. Lister le contenu du répertoire `$HOME/.ssh/` puis afficher le contenu du fichier `id_ecdsa.pub`

Ecrire la commande : `cd $HOME/.ssh/`

Le fichier de la clé est par défaut camouflé, à l'aide de la commande `ls -la` je vais pouvoir voir les fichiers cachés

```
etusio@clissh:~$ cd //home/etusio/.ssh
etusio@clissh://home/etusio/.ssh$ cd
etusio@clissh:~$ ls -la /home/etusio/.ssh
total 20
drwx----- 2 etusio etusio 4096 déc. 18 09:07 .
drwxr-xr-x 15 etusio etusio 4096 déc. 18 08:55 ..
-rw----- 1 etusio etusio 557 déc. 18 09:07 id_ecdsa
-rw-r--r-- 1 etusio etusio 175 déc. 18 09:07 id_ecdsa.pub
-rw-r--r-- 1 etusio etusio 1 nov. 28 11:01 known_hosts
etusio@clissh:~$ ls -la /home/etusio/.ssh
```

La commande `cat` permet d'afficher le contenu d'un fichier :

```
etusio@clissh:~$ cd /$HOME/.ssh/
etusio@clissh://home/etusio/.ssh$ cat id_ecdsa.pub
ecdsa-sha2-nistp256 AAAAE2VjZHNhLXNoYTItbmlzdHAyNTYAAAIbmlzdHAyNTYAAABBB
Np0bmrYIKn/l1BMucz8aVNe9amWgHgoY5E2WeB4bB8HyfWp6iCu9z66G54znWVBP4p2sIV3je
PCc3YcRetRvNk= etusio@clissh
etusio@clissh://home/etusio/.ssh$
```

`echo` permet de voir le chemin d'un dossier

```
etusio@clissh://home/etusio/.ssh$ echo $HOME
/home/etusio
etusio@clissh://home/etusio/.ssh$ cd
etusio@clissh:~$ echo $HOME
/home/etusio
etusio@clissh:~$ echo $HOME
/home/etusio
```

Nous constatons après l'exécution de la copie et de celui présent, dans le fichier ecdsa.pub

Faire la commande : "sudo service ssh restart" sur le serveur Debian

```
etusio@srvssh:~$ sudo service ssh restart
[sudo] Mot de passe de etusio :
etusio@srvssh:~$ _
```

Je rafraîchis ma clé, avec le même mot de passe :

```
+---[ECDSA 256]---+
|
|      o
|    . o.=..
|    o  S*=o.
|    . +.*** .
|    + 0=*o+  E
|    + .oB+B =
|    o..+@*= o.
|
+-----[SHA256]-----+
```

```
Enter file in which to save the key (/home/
/home/etusio/.ssh/id_ecdsa already exists.
Overwrite (y/n)? y
```

```
etusio@clissh:~$ ssh-copy-id -i ~/.ssh/id_ecdsa.pub etusio@srvssh.local.sio.fr
/usr/bin/ssh-copy-id: INFO: Source of key(s) to be installed: "/home/etusio/.ssh/id_ecdsa.pub"
/usr/bin/ssh-copy-id: INFO: attempting to log in with the new key(s), to filter out any that
are already installed
/usr/bin/ssh-copy-id: ERROR: ssh: Could not resolve hostname srvssh.local.sio.fr: Temporary f
ailure in name resolution
```

Si la copie ne fonctionnait pas c'est, la raison est que le serveur n'est sûrement pas ouvert pour faire la copie de clé :

```

etusio@clissh:~$ //home/etusio/.ssh
bash: //home/etusio/.ssh : est un dossier
etusio@clissh:~$ ssh-copy-id -i ~/.ssh/id_ecdsa.pub etusio@srvssh.local.sio.fr
/usr/bin/ssh-copy-id: INFO: Source of key(s) to be installed: "/home/etusio/.ssh/id_ecdsa.pub"
/usr/bin/ssh-copy-id: INFO: attempting to log in with the new key(s), to filter out any that
are already installed
/usr/bin/ssh-copy-id: ERROR: ssh: Could not resolve hostname srvssh.local.sio.fr: Temporary failure in name resolution

```

J'ai encore écrit la commande de "copy" sur la machine cliente. Et je peux effectuer le test sur le serveur : **cat .ssh/authorized_keys**

```

etusio@srvssh:~$ cat .ssh/authorized_keys
cat: .ssh/authorized_keys: Aucun fichier ou dossier de ce type
etusio@srvssh:~$ cat .ssh/authorized_keys
ecdsa-sha2-nistp256 AAAAE2VjZHNhLXNoYTItbmlzdHAyNTYAAAAIbmlzdHAyNTYAAABBBBCP7SNXzs3qS5IyjhIh/byr858+m
eaMzMUNaArvie0hAiW3n0jGLcm2tn1fRSXTLtBoxVZmpXW3q3MyKgF0KSJw= etusio@clissh
etusio@srvssh:~$

```

Q24. Sur la machine cliente clissh.local.sio.fr, modifier les droits du fichier ~/.ssh/id_ecdsa (droits initiaux : 600 etusio:etusio) qui contient votre clé privée en 644. Se connecter sur le serveur distant srvssh.local.sio.fr qui contient la clé publique. Que se passe-t-il ? Pourquoi ?

Je me déplace sur le répertoire **.ssh**

La commande **ls -l** permet de voir ce que j'ai dans le dossier **.ssh**

```

etusio@clissh:~$ cd /home/etusio/.ssh
etusio@clissh:~/.ssh$ ls -l
total 12
-rw----- 1 etusio etusio 557 déc. 18 10:53 id_ecdsa
-rw-r--r-- 1 etusio etusio 175 déc. 18 10:53 id_ecdsa.pub
-rw-r--r-- 1 etusio etusio 445 déc. 18 10:57 known_hosts

```

Maintenant je dois modifier les droits de la clé privée et publique :

clé publique → `chmod 600 id_ecdsa.pub`

clé privée → `chmod 644 id_ecdsa`

```
etusio@clissh:~/.ssh$ chmod 600 id_ecdsa.pub
etusio@clissh:~/.ssh$ chmod 644 id_ecdsa
etusio@clissh:~/.ssh$ ls -l
total 12
-rw-r--r-- 1 etusio etusio 557 déc. 18 10:53 id_ecdsa
-rw----- 1 etusio etusio 175 déc. 18 10:53 id_ecdsa.pub
-rw-r--r-- 1 etusio etusio 445 déc. 18 10:57 known_hosts
```

Si on compare les droits du screen précédent, on peut voir que `id_ecdsa` est en lire écrire pour

Connexion à distance sur le serveur depuis la machine cliente :

commande : `ssh etusio@srvssh.local.sio.fr`

```
etusio@clissh:~/.ssh$ ssh etusio@srvssh.local.sio.fr
@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
@                WARNING: UNPROTECTED PRIVATE KEY FILE!                @
@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
Permissions 0644 for '/home/etusio/.ssh/id_ecdsa' are too open.
It is required that your private key files are NOT accessible by others.
This private key will be ignored.
Load key "/home/etusio/.ssh/id_ecdsa": bad permissions
etusio@srvssh.local.sio.fr's password:
Permission denied, please try again.
etusio@srvssh.local.sio.fr's password:
Linux srvssh 4.19.0-8-amd64 #1 SMP Debian 4.19.98-1+deb10u1 (2020-04-27) x86_64
```

Un message d'erreur apparaît dans le navigateur du terminal serveur indiquant que la clé privée du chiffrement, ne doit surtout pas être

accessible à quelqu'un d'autre et que les droits d'accès mise en oeuvre sont trop permissifs

Déconnectez vous du serveur en tapant la commande : exit

Q25. Rétablir les droits de ~/.ssh/id_ecdsa sur le poste client. Maintenant sur le serveur distant, afficher les droits d'accès appliqués au fichier ~/.ssh/authorized_keys.

Les droits d'accès du répertoire "authorized_keys" est **600** tout comme la clé privée.

```
etusio@srvssh:~$ ls -l ~/.ssh
total 16
-rw----- 1 etusio etusio 175 déc. 18 10:57 authorized_keys
-rw----- 1 etusio etusio 1823 nov. 28 10:04 id_rsa
-rw-r--r-- 1 etusio etusio 395 nov. 28 10:04 id_rsa.pub
-rw-r--r-- 1 etusio etusio 1 nov. 28 10:08 known_hosts
```

Remarque -rw voie de lecture et d'écriture pour le propriétaire, ici etusio, tous les fichiers authorized_keys. Aucune autorisation pour le groupe ou les autres utilisateurs du système.

Q26. Puis, modifiez les droits de ~/.ssh/authorized_keys en 666. Se déconnecter puis se reconnecter. Vérifier si un changement est apparu ou non et tenter d'expliquer pourquoi (ne pas oublier de rétablir les droits d'origine ensuite).

Je ne peux pas accéder au fichier →

```
etusio@srvssh:~$ chmod 666 authorized_keys
chmod: impossible d'accéder à 'authorized_keys': Aucun fichier ou dossier de ce type
etusio@srvssh:~$ sudo chmod 666 authorized_keys
[sudo] Mot de passe de etusio :
chmod: impossible d'accéder à 'authorized_keys': Aucun fichier ou dossier de ce type
etusio@srvssh:~$ chmod 666 authorized_keys
chmod: impossible d'accéder à 'authorized_keys': Aucun fichier ou dossier de ce type
```

La connexion depuis le client est impossible à nouveau. En effet, le fichier qui contient les clés publiques autorisées à se connecter avec l'utilisateur en question ne doit pas être accessible à tous sur le système.

Le service ssh considère donc ces droits incorrects pour autoriser une connexion (visible dans les fichiers annales, /var/log/auth.log, bad ownership or modes. En effet, même si l'authentificateur est généré par le client est valide, cela ne nous permet en aucun cas à l'attaque de récupérer la clé privée du client.

Ainsi l'utilisation de ce type d'authentification permet de prévenir des attaques ssh mitm.

Q27. En analysant la connexion par clés, proposer une hypothèse de fonctionnement de cette nouvelle forme d'authentification. Expliquer ce qui différencie une connexion par mot de passe d'une connexion par clé de chiffrement.

Ce qui différencie une connexion par mot de passe et d'une connexion par clé de chiffrement :

Le MDP peut être partagé et retrouvé facilement par un tiers tandis que la connexion par clé de chiffrement est complexe et la clé privée secrète.

L'authentification par clé est une forme qui permet à l'utilisateur d'apporter la preuve de son identité. Aussi une connexion par mot de passe est une forme d'identification ou on utilise un "login" et un "mdp". La connexion par clé de chiffrement, elle, intervient après la phase dite "identification". Cette connexion permet de vérifier réellement l'identité d'un tiers. La connexion par clé de chiffrement d'applique sur l'utilisation d'une paire de clé (publique et privée) pour renforcer la sécurité.

2. Utilisation de ssh-agent

Je n'arrive pas à entrer le mot de passe de la clé de chiffrement, du à un défaut de sécurisation, je ne peux y accéder.

```
etusio@clissh:~$ exec ssh-agent $SHELL
etusio@clissh:~$ sudo ssh-add
Could not open a connection to your authentication agent.
etusio@clissh:~$ ssh-add
@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
@                WARNING: UNPROTECTED PRIVATE KEY FILE!                @
@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
Permissions 0644 for '/home/etusio/.ssh/id_ecdsa' are too open.
It is required that your private key files are NOT accessible by others.
This private key will be ignored.
```

Je rafraichis les droits d'accès de la clé privée :

```
etusio@clissh:~$ chmod 600 ~/.ssh/id_ecdsa
etusio@clissh:~$ ls -la ~/.ssh/id_ecdsa
-rw----- 1 etusio etusio 557 déc. 19 09:20 /home/etusio/.ssh/id_ecdsa
```

Je tente de me reconnecter sur la clé de chiffrement, on me demande enfin mon mot de passe.

```
etusio@clissh:~$ ssh-add ~/.ssh/id_ecdsa
Enter passphrase for /home/etusio/.ssh/id_ecdsa:
Identity added: /home/etusio/.ssh/id_ecdsa (etusio@clissh)
```

Q28.Suite à la mise en place de cette authentification par clés de chiffrement, tenter à nouveau de simuler une attaque MITM entre le client et le serveur SSH. Quel résultat obtenez-vous ? Pourquoi ?

Simulation d'attaque sur Kali :

```
etusio@kali:~$ git clone https://github.com/ktux/ssh-mitm
Clonage dans 'ssh-mitm' ...
fatal: impossible d'accéder à 'https://github.com/ktux/ssh-mitm/' : Could not resolve host: github.com
```


Commandes de simulation :

```
git clone https://github.com/ktux/ssh-mitm
```

```
cd ssh-mitm/
```

```
/ssh-mitm$ export LANG=en_US.utf-8
```

```
/ssh-mitm$ sudo ./install.sh
```

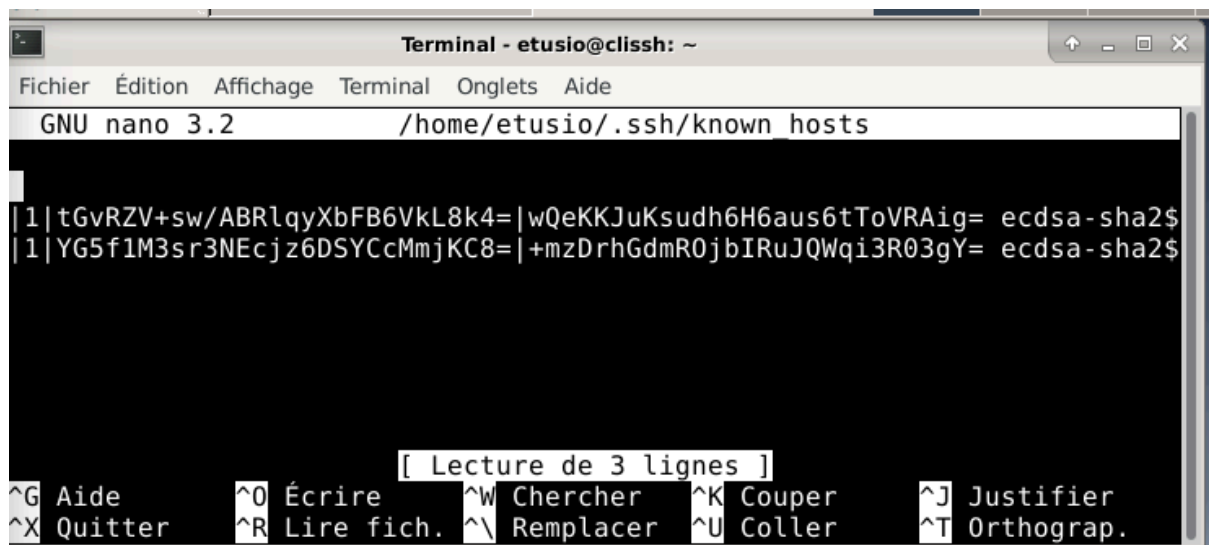
```
sudo ettercap -i eth0 - -m arp/192.168.56.10// 192.168.65.11//
```

Après avoir mis en place cette méthode de participation, l'attaquant n'est plus en mesure de pouvoir récupérer l'identifiant de mot de passe utilisateur. Dans l'hypothèse où l'attaquant disposerait du bon compte nominatif et de l'empreinte de la clé publique du client, cela pourrait éventuellement entraîner une connexion du client sur la machine de l'attaquant mais cela n'aurait aucun intérêt sur ce dernier. En effet, même si l'authentificateur généré par le client est valide, cela ne permet en aucun cas de récupérer la clé privée du client.

Ainsi l'utilisation de ce type d'authentification permet de prévenir les attaques ssh-mitm.

Effacer le contenu du fichier known_hosts :

Regarder le contenu du fichier known_hosts : `nano ~/.ssh/known_hosts`

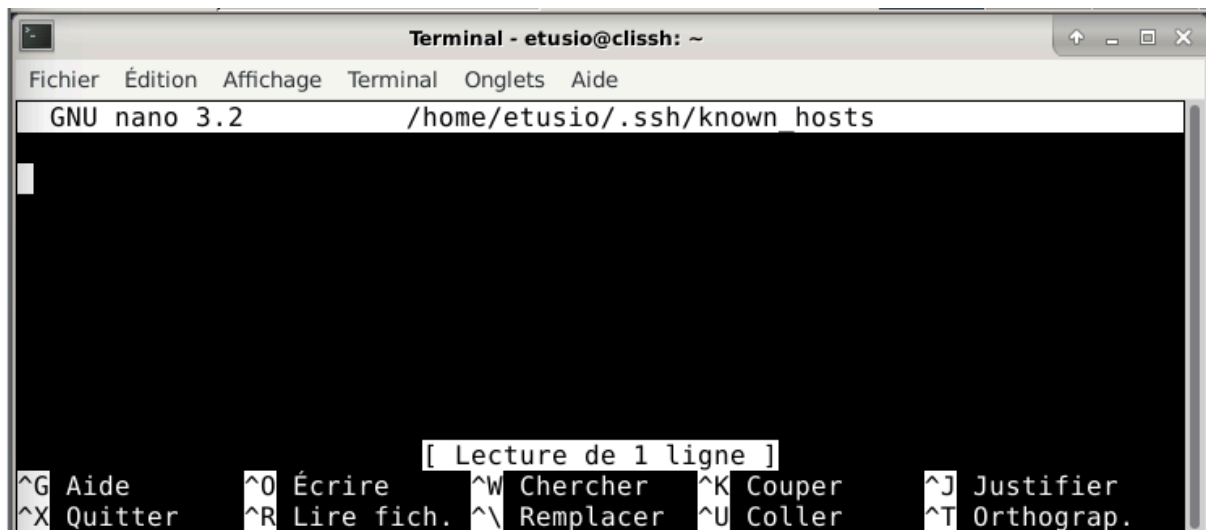


```
Terminal - etusio@clissh: ~
Fichier  Édition  Affichage  Terminal  Onglets  Aide
GNU nano 3.2 /home/etusio/.ssh/known_hosts

|1|tGvRZV+sw/ABRLqyXbFB6VkL8k4=|wQeKKJuKsudh6H6aus6tToVRAig= ecdsa-sha2$
|1|YG5f1M3sr3NEcjz6DSYCcMmjKC8=|+mzDrhGdmR0jbIRuJQWqi3R03gY= ecdsa-sha2$

[ Lecture de 3 lignes ]
^G Aide      ^O Écrire    ^W Chercher  ^K Couper    ^J Justifier
^X Quitter   ^R Lire fich.^_ Remplacer  ^U Coller    ^T Orthograp.
```

Pour supprimer les données : `echo > ~/.ssh/known_hosts`



3. Faciliter la vérification de l'identité du serveur SSH avant la première connexion

Comment voir si le serveur a la même clé de le client : **ssh-keygen -l**

etusio@srvssh :

```
etusio@srvssh:~$ ssh-keygen -I
option requires an argument -- I
usage: ssh-keygen [-q] [-b bits] [-t dsa | ecdsa | ed25519 | rsa]
                [-N new_passphrase] [-C comment] [-f output_keyfile]
ssh-keygen -p [-P old_passphrase] [-N new_passphrase] [-f keyfile]
ssh-keygen -i [-m key_format] [-f input_keyfile]
ssh-keygen -e [-m key_format] [-f input_keyfile]
ssh-keygen -y [-f input_keyfile]
ssh-keygen -c [-P passphrase] [-C comment] [-f keyfile]
ssh-keygen -l [-v] [-E fingerprint_hash] [-f input_keyfile]
ssh-keygen -B [-f input_keyfile]
ssh-keygen -D pkcs11
ssh-keygen -F hostname [-f known_hosts_file] [-l]
ssh-keygen -H [-f known_hosts_file]
ssh-keygen -R hostname [-f known_hosts_file]
ssh-keygen -r hostname [-f input_keyfile] [-g]
ssh-keygen -G output_file [-v] [-b bits] [-M memory] [-S start_point]
ssh-keygen -T output_file -f input_file [-v] [-a rounds] [-J num_lines]
                [-j start_line] [-K checkpt] [-W generator]
ssh-keygen -s ca_key -I certificate_identity [-h] [-U]
                [-D pkcs11_provider] [-n principals] [-O option]
                [-V validity_interval] [-z serial_number] file ...
ssh-keygen -L [-f input_keyfile]
ssh-keygen -A
ssh-keygen -k -f krl_file [-u] [-s ca_public] [-z version_number]
                file ...
ssh-keygen -Q -f krl_file file ...
```

etusio@clissh :

```
etusio@clissh:~$ ssh-keygen -I
option requires an argument -- I
usage: ssh-keygen [-q] [-b bits] [-t dsa | ecdsa | ed25519 | rsa]
                  [-N new_passphrase] [-C comment] [-f output_keyfile]
ssh-keygen -p [-P old_passphrase] [-N new_passphrase] [-f keyfile]
ssh-keygen -i [-m key_format] [-f input_keyfile]
ssh-keygen -e [-m key_format] [-f input_keyfile]
ssh-keygen -y [-f input_keyfile]
ssh-keygen -c [-P passphrase] [-C comment] [-f keyfile]
ssh-keygen -l [-v] [-E fingerprint_hash] [-f input_keyfile]
ssh-keygen -B [-f input_keyfile]
ssh-keygen -D pkcs11
ssh-keygen -F hostname [-f known_hosts_file] [-l]
ssh-keygen -H [-f known_hosts_file]
ssh-keygen -R hostname [-f known_hosts_file]
ssh-keygen -r hostname [-f input_keyfile] [-g]
ssh-keygen -G output_file [-v] [-b bits] [-M memory] [-S start_point]
ssh-keygen -T output_file -f input_file [-v] [-a rounds] [-J num_lines]
```

La commande étant : `ssh-keygen -l -f /etc/ssh/ssh_host_ecdsa_key.pub`

```
etusio@srvssh:~$ ssh-keygen -l -f /etc/ssh/ssh_host_ecdsa_key.pub
256 SHA256:lPIxEKxsYHPP3i7iMiZZXLYUoW9viLwSfF39MNoWlM4 root@srvssh (ECDSA)
```

Depuis la machine cliente, on constate que les clés sont similaires

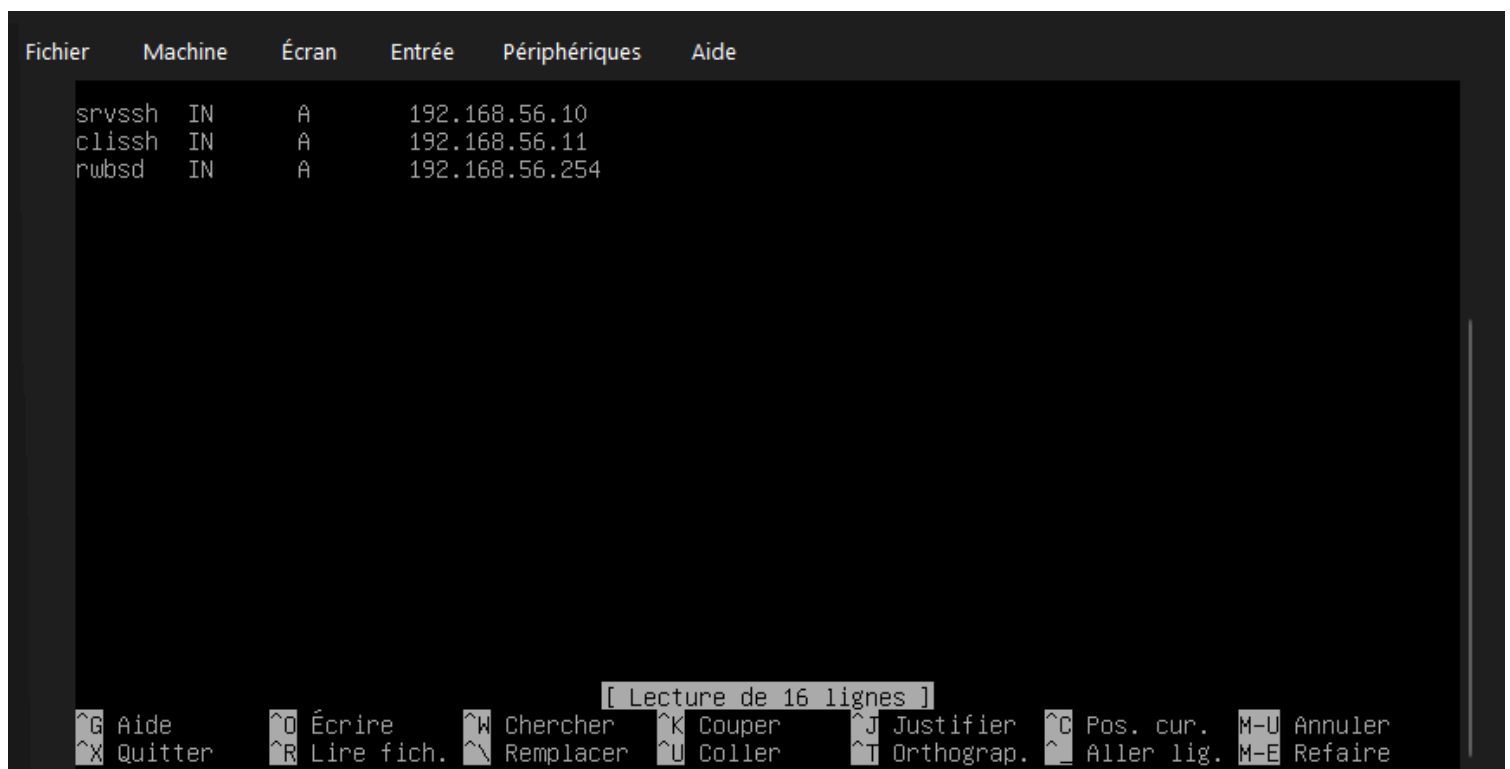
```
etusio@clissh:~$ ssh etusio@srvssh.local.sio.fr
The authenticity of host 'srvssh.local.sio.fr (192.168.56.10)' can't be established.
ECDSA key fingerprint is SHA256:lPIxEKxsYHPP3i7iMiZZXLYUoW9viLwSfF39MNoWlM4.
Are you sure you want to continue connecting (yes/no)? yes
Warning: Permanently added 'srvssh.local.sio.fr,192.168.56.10' (ECDSA) to the list of known hosts.
ssh_dispatch_run_fatal: Connection to 192.168.56.10 port 22: Broken pipe
```

```
etusio@clissh:~$ ssh-keygen -l -f /etc/ssh/ssh_host_ecdsa_key.pub
256 SHA256:N8S208r0j5DsKyt78GgLDzdXuQlCdEv0Ft8B3ySE5uo root@clissh (ECDSA)
```

Q29.Expliquer l'intérêt de cette démarche.

La démarche qui consiste à valider manuellement la clé et l'enregistrer dans le DNS de l'entreprise permet de gagner considérablement en temps. Une fois validé, cette clé sera automatiquement reconnu par tous les postes de l'entreprise, simplifiant ainsi le processus d'authentification

```
etusio@srvssh:~$ sudo ssh-keygen -r srvssh
[sudo] Mot de passe de etusio :
srvssh IN SSHFP 1 1 6fa8aeb8227f09bc4f8c8afe08245a8c7718d324
srvssh IN SSHFP 1 2 ef3c9989952d8591cc761f1b26a658354a93230dae730edbe42732bf0a1219a8
srvssh IN SSHFP 3 1 4c16f0e9298469630445a24c57c342b15540253f
srvssh IN SSHFP 3 2 94f23110ac6c6073cfde2ee23226595cb614a16f6f88bc127c5dfd30da1694ce
srvssh IN SSHFP 4 1 7ef6631af09e0585ff8c3169255189195f7b7ddf
srvssh IN SSHFP 4 2 88efa67bb697dafdde1c18bee5c76218e8c78803bfa6b3e093c260cf56bd9f6e
```



La commande `service bind9 reload` permet d'actualiser le serveur.
`VerifyHostKeyDNS`, procède à la vérification de connexion sur le réseau.
Il est rattaché au serveur `srvssh.local.sio.fr`

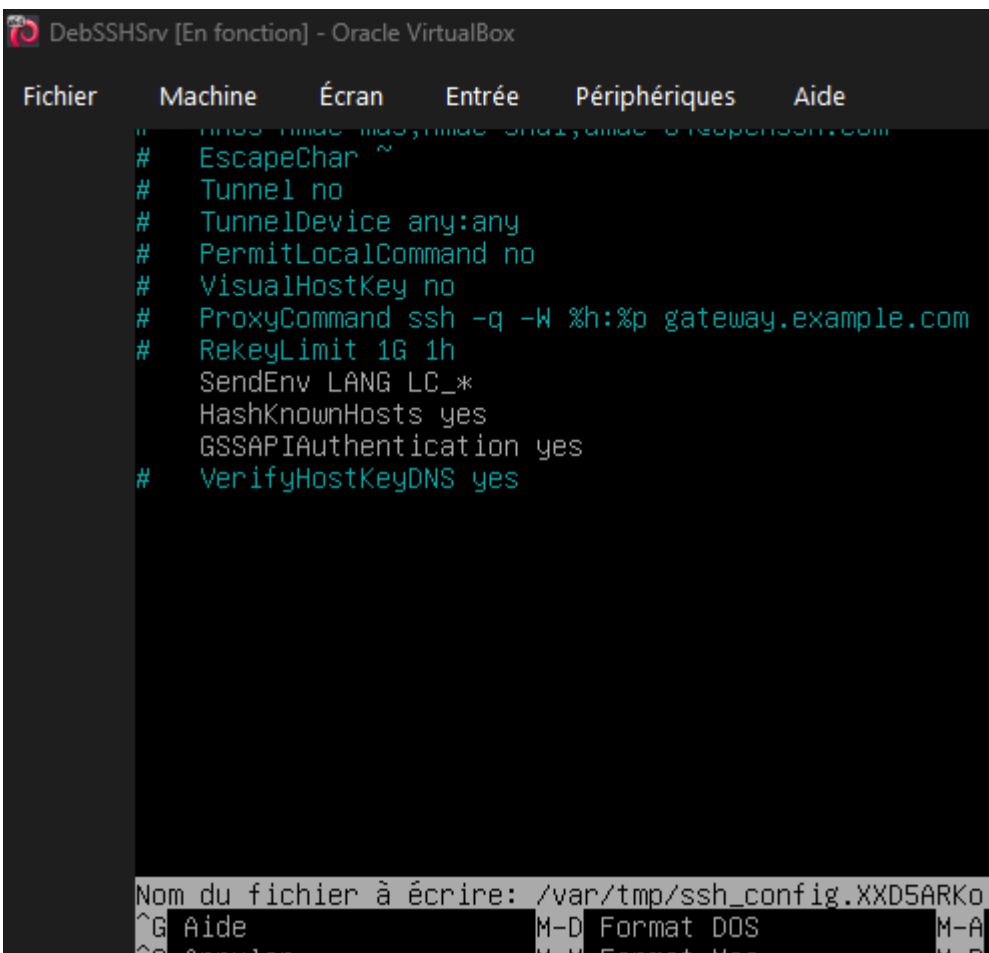
```
sudoedit: /var/named/db.local.sio.fr non modifié
etusio@srvssh:~$ sudo service bind9 reload
etusio@srvssh:~$ ssh -o VerifyHostKeyDNS=true etusio@srvssh.local.sio.fr
The authenticity of host 'srvssh.local.sio.fr (127.0.1.1)' can't be established.
ECDSA key fingerprint is SHA256:1PIxEKxsYHPP3i7iMiZZXLYUoW9viLwSfF39MNoW1M4.
No matching host key fingerprint found in DNS.
Are you sure you want to continue connecting (yes/no)? yes
Warning: Permanently added 'srvssh.local.sio.fr' (ECDSA) to the list of known hosts.
etusio@srvssh.local.sio.fr's password:
Linux srvssh 4.19.0-8-amd64 #1 SMP Debian 4.19.98-1+deb10u1 (2020-04-27) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Fri Dec 19 10:46:00 2025
```

Ajouter la fonctionnalité de vérification de l'hôte manuellement sur le script "`sudoedit /etc/ssh/ssh_config`", `sudoedit` permet avec les droits de

root de modifier le script au-delà des permissions.



```
DebSSHSrv [En fonction] - Oracle VirtualBox
Fichier  Machine  Écran  Entrée  Périphériques  Aide
# HostName srvssh.local.sio.fr
# EscapeChar ~
# Tunnel no
# TunnelDevice any:any
# PermitLocalCommand no
# VisualHostKey no
# ProxyCommand ssh -q -W %h:%p gateway.example.com
# RekeyLimit 1G 1h
# SendEnv LANG LC_*
# HashKnownHosts yes
# GSSAPIAuthentication yes
# VerifyHostKeyDNS yes

Nom du fichier à écrire: /var/tmp/ssh_config.XXD5ARKo
^G Aide M-D Format DOS M-A
^C Annuler M-M Format Mac M-P
```

Q30. Mettre en œuvre les préconisations suivantes tirées des recommandations pour un usage sécurisé de SSH publié par l'ANSSI :

4. Amélioration de la sécurité du service OpenSSH sur le serveur

1. Vérifier que les clés privées de chiffrement présentes dans le répertoire /etc/ssh/ appartiennent à l'utilisateur root en lecture-écriture seulement ;

Toutes les clés privées sont en 600

```
etusio@srvssh:~$ ls -l ~/.ssh/
total 16
-rw----- 1 etusio etusio 350 déc. 19 09:22 authorized_keys
-rw----- 1 etusio etusio 1823 nov. 28 10:04 id_rsa
-rw-r--r-- 1 etusio etusio 395 nov. 28 10:04 id_rsa.pub
-rw-r--r-- 1 etusio etusio 223 déc. 19 11:21 known_hosts
```

2. Version 2 du protocole SSH :

Réponses sur ce lien :

 GABIN Floralyn - Amélioration de la sécurité du service OpenSSH ...

5. Durcissement des algorithmes de chiffrement utilisés entre le client et le serveur SSH

Q31. Expliquer dans une définition succincte ce qu'est l'état de l'art dans le domaine de la cybersécurité.

Sert à aider les usagers pour tester et améliorer leur cyber-résilience face aux cyberattaques. Elle peut servir à prendre vie par l'IA ou le type EDR (indicateur d'attaque). Afin d'en faire un système automatisé d'un dessin qui prend vie et bloque les menaces.

L'état de l'art en cybersécurité représente le niveau le plus avancé de connaissance, de technologie et de pratique actuelle visant à prévenir, détecter et contrer les menaces informatiques. Il garantit ainsi une protection efficace des systèmes, des réseaux et des données contre les attaques potentielles.

Q32.Définir ce qu'est le principe de Kerckhoffs.

Le principe de Kerckhoffs repose sur la solidité de l'algorithme et le secret de la clé. La méthode de chiffrement doit être publique et robuste : elle peut tomber entre les mains d'un hacker sans aucun danger. Le plus important est de sécuriser la clé, car tant qu'elle est hors de portée, le système reste inviolable.

Le principe de Kerckhoffs pose le constat que la qualité d'un algorithme cryptographique repose sur sa robustesse et sa fiabilité mathématique et non sur le fait qu'il soit inconnu de l'ennemi

Q33.Selon ce principe, pourquoi est-il pertinent de choisir des algorithmes cryptographiques connus et respectant l'état de l'art ?

Il est pertinent de choisir des algorithmes connus car ils bénéficient de l'analyse publique des experts, garantissant leur robustesse. Un algorithme standard respectant l'état de l'art est éprouvé contre les attaques, contrairement à un système secret dont les failles n'ont pas encore été révélées.